ProEm Standards Programming Guide

Receive and transmit programs for sixteen standards, all tested

Paul Vincent Bezzina

11 October 2026

Abstract

This guide gives the program for each of sixteen communication standards, for receiving and for transmitting, as instruction bytes ready to send to ProEm. Every listing is the exact program used by the pin-level test `sim/t_standards.cpp`, which drives or models the real line for each standard and checks what ProEm receives and what it puts on the wire; all 42 checks pass. For each standard the guide explains the frame, how it maps onto ProEm's blocks, what the host gets back and what the host still has to do. A final section lists the limits the first round of tests uncovered and how ISA v0.7 removed them.

Contents

1 How to read this guide **3**

2 Summary **4**

3 RS-232, RS-422 and RS-485 **4**

4 MIDI **5**

5 NMEA 0183 **5**

6 PROFIBUS DP **6**

7 Modbus RTU **6**

8 DMX512 **7**

9 LIN **8**

10 K-Line (ISO 14230 / KWP2000) **9**

11 SDI-12 **11**

12 SMBus and PMBus **12**

13 MDIO (Ethernet PHY management) **13**

14 SD card in SPI mode **14**

15 3-wire SPI **15**

16 DALI **15**

17 CAN 2.0B and NMEA 2000 **16**

18 RC5 infrared **17**

19 Limits found by the first round of tests, and the fixes **18**

20 Running the tests **18**

1 How to read this guide

Bytes. Listings show instruction bytes in hex, one instruction per line, followed by a short note. A program is sent once; the per-frame part (`DATA` and `RUN`) is sent for every frame. Opcodes for channel A are `x0`–`x7`; for channel B they are 8 higher (`A8` instead of `A0`, `C8` for `DATA` of length 1, and so on). The listings use the channel the test used.

Pins. All programs use the Tiny Tapeout pin map: channel A drives `uio[0]` (data), `uio[1]` (pair) and `uio[2]` (clock); channel B drives `uio[3]`, `uio[4]` and `uio[5]`; `uio[6]` and `uio[7]` are inputs only; the select outputs are `uo[6]` (A) and `uo[7]` (B). Every `uio` pad is read back, so a pin number in a program is the pad number.

Timing. `DIV` values are for the 50 MHz system clock: clocks per bit for ASYNC and RECOVER, clocks per half period for GENERATE. A different clock scales them.

Return messages. Received data comes back as `8L vv` (chain A, length L) or `AL vv` (chain B); `8E/AE` is a CRC result; `8F/AF` is the status at the end of each frame, `01` meaning complete and `02` meaning aborted. In the tables below a received entry is written {`L,vv`}.

Five patterns used throughout.

1. **UART receive, one frame per character.** `START` reads 1 bit = 0 (the start bit) and `MAIN` reads the data, parity and stop bits. The chain re-arms after every character, so any idle time between characters is fine, and the bit timer re-centres on every edge.
2. **UART transmit, one frame per character.** `START` writes the start bit; `MAIN` writes the rest from two `DATA` entries (data bits, then parity and stop bits). With plain `RUN` the chain starts a new character whenever data is waiting. The host keeps the 8-entry FIFO topped up and watches `HOST_FULL`.
3. **UART receive with a framing check.** `START` reads the start bit, a capture-any header takes the data, a header compares the stop bit with 1, and `END` watches for a 1 on the line. A break (a long 0) fails the stop-bit compare, the frame aborts with status `02`, and `START` then waits until the line is idle again. This is how a hardware UART detects a break, and it makes DMX512 reception independent of the break and mark lengths.
4. **Fixed fields in headers, payload in `MAIN`.** Wake-up patterns, breaks, preambles, addresses and commands are header values written once with the program; only the payload changes per frame.
5. **Status when the line is free.** A write frame normally completes as soon as its last bits are queued. Adding the rule `F0 08 00 00` (or `F8 08 00 00` on B: read-back source, both actions "keep") holds the frame open until its last bit has left the pin, so the status tells the host exactly when the line is free. The K-Line and SDI-12 programs use it to start the next step on time.
6. **Seek to resynchronise.** A read header with `SEEK` ignores incoming bits until its first bit appears, then compares the rest as usual, so a frame can wait out a gap of any length in the middle; a 1-bit seeking header with value 0 waits for the next start bit (ISA v0.7).

2 Summary

Standard	Line	DIV	Channels	Checks
RS-232 / RS-422 / RS-485	115.2 kbit/s 8N1	434	A rx, B tx	2
MIDI	31.25 kbit/s 8N1	1600	A rx, B tx	2
NMEA 0183	4800 8N1	10417	A rx, B tx	2
PROFIBUS DP	500 kbit/s 8E1	100	A rx, B tx	2
Modbus RTU	19200 8E1, CRC on chip	2604	A rx, B tx	5
DMX512	250 kbit/s 8N2	200	A rx, B tx	2
LIN	19200, break + sync	2604	A	2
K-Line (ISO 14230)	10400, fast and 5-baud init	4808 / 62500 / 2500000	A + B	4
SDI-12	1200 7E1, inverted	41667	A + B	2
SMBus / PMBus	100 kHz, PEC on chip	250	A	4
MDIO (IEEE 802.3 cl. 22)	2.5 MHz	10	A	3
SD card (SPI mode)	400 kHz, CRC-7	63	A + B	2
3-wire SPI	1 MHz	25	A	2
DALI	1200, Manchester	41666	A rx, B tx	3
CAN 2.0B / NMEA 2000	250 kbit/s, 29-bit ID	200	A rx, B tx	3
RC5 infrared	1.778 ms Manchester	88900	A rx, B tx	2

Checks are the number of pass/fail lines each test prints (42 in all). `bash scripts/run.sh` runs them with everything else; Section 20 shows how to run one standard.

3 RS-232, RS-422 and RS-485

All three are the same asynchronous UART frame (a 0 start bit, data bits LSB first, optional parity, 1 or 2 stop bits) on different electrical layers: RS-232 uses ±12 V single-ended levels, RS-422 a differential pair with one driver, RS-485 a differential pair shared by many nodes, half duplex. ProEm handles the logic level; a MAX3232 or an RS-485 transceiver does the rest.

115 200 bit/s, 8N1, `DIV` = 434. RS-485 transceiver: RO to `uio[0]`, DI to `uio[3]`, DE (and /RE) to `uo[7]`, the select output of chain B, active high.

Receive (chain A, uio[0])

```
A0 00 00         LINE: NRZ on pin 0
B0 40 00 01 B2   CLOCK: ASYNC, 434 clocks per bit
10 01 00 00      START: read 1 bit = 0 (start bit) on pin 0
30 00 00 09      MAIN: read 9 bits (data, parity, stop)
70               RUN: armed, re-arms after every character
```

Transmit (chain B, uio[3]) with the driver enable on uo[7]

```
A8 03 00         LINE: NRZ, read-back pin 3
B8 40 00 01 B2   CLOCK: ASYNC, 434 clocks per bit
D8 62            OUT: push-pull while sending, idle 1, select active high
18 81 00 00      START: write 1 bit = 0 (start bit)
38 43 00 09      MAIN: write 9 bits from DATA
```

Per character

```
CF 47   DATA {8,'G'}: the data bits
C8 01   DATA {1,1}: the stop bit
78      RUN once: a character starts whenever data waits
```

What comes back. Each character returns {`8,data`} and {`1,stop`} followed by status `01`. A stop bit of 0 is a framing error.

Host work. Group characters into messages. For RS-485, the select output is active from the first bit of each character until its stop bit has left the pin, which is exactly the driver-enable window; nothing else is needed.

Tested. Six bytes received (`01 10 A5 5A FF 00`). Four bytes sent and decoded on the line; the test checks that DE is high on every clock the data pad is driven (17,364 clocks) and low at the end.

4 MIDI

MIDI 1.0 is a 31 250 bit/s UART on a 5 mA current loop. Messages are a status byte (bit 7 set) followed by one or two data bytes, for example Note On `9n kk vv`.

31 250 bit/s, 8N1, `DIV` = 1600 (exact). MIDI IN through the usual optocoupler to `uio[0]`; MIDI OUT from `uio[3]` through a buffer and 220 Ω resistors.

Receive (chain A)

```
A0 00 00         LINE: NRZ on pin 0
B0 40 00 06 40   CLOCK: ASYNC, 1600 clocks per bit
10 01 00 00      START: read 1 bit = 0 (start bit) on pin 0
30 00 00 09      MAIN: read 9 bits (data, parity, stop)
70               RUN: armed, re-arms after every character
```

Transmit (chain B), push-pull always so the line idles high

```
A8 03 00         LINE: NRZ, read-back pin 3
B8 40 00 06 40   CLOCK: ASYNC, 1600 clocks per bit
D8 E0            OUT: push-pull always, idle 1
18 81 00 00      START: write 1 bit = 0 (start bit)
38 43 00 09      MAIN: write 9 bits from DATA
```

What comes back. `{8,byte},{1,1}` and status `01` per byte.

Host work. Parse status and running status; the chip only frames bytes.

Tested. Note On `90 3C 64` received; Note Off `80 3C 00` sent and decoded.

5 NMEA 0183

Marine and GPS instruments send ASCII sentences such as `$GPGLL,...*hh<CR><LF>`, where `hh` is the XOR of every character between `$` and `*`. The line is a 4800 bit/s UART (RS-422 levels on real equipment).

4800 bit/s, 8N1, `DIV` = 10417 (0.003 % fast).

Receive (chain A)

```
A0 00 00         LINE: NRZ on pin 0
B0 40 00 28 B1   CLOCK: ASYNC, 10417 clocks per bit
10 01 00 00      START: read 1 bit = 0 (start bit) on pin 0
30 00 00 09      MAIN: read 9 bits (data, parity, stop)
70               RUN: armed, re-arms after every character
```

Transmit (chain B)

```
A8 03 00         LINE: NRZ, read-back pin 3
B8 40 00 28 B1   CLOCK: ASYNC, 10417 clocks per bit
D8 E0            OUT: push-pull always, idle 1
```

```
18 81 00 00      START: write 1 bit = 0 (start bit)
38 43 00 09      MAIN: write 9 bits from DATA
```

What comes back. One {8,char},{1,1} pair and status per character.

Host work. Assemble sentences from $ to <LF> and check or add the XOR checksum (31 for the GLL example).

Tested. A 41-character GLL sentence received at the real 4800 bit/s, and a proprietary sentence $PPROE,1,TEST*7F sent and decoded with its checksum.

6 PROFIBUS DP

PROFIBUS DP sends 11-bit characters (start, 8 data, even parity, stop) on RS-485. The short telegram SD1 is 10 DA SA FC FCS 16, with FCS the 8-bit sum of DA, SA and FC.

500 kbit/s (one of the standard rates), 8E1, DIV = 100. Faster rates (1.5 to 12 Mbit/s) need DIV below 34; 1.5 Mbit/s works with DIV = 33 (1 % fast).

Receive (chain A): MAIN reads 10 bits, data + parity + stop

```
A0 00 00         LINE: NRZ on pin 0
B0 40 00 00 64   CLOCK: ASYNC, 100 clocks per bit
10 01 00 00      START: read 1 bit = 0 (start bit) on pin 0
30 00 00 0A      MAIN: read 10 bits (data, parity, stop)
70               RUN: armed, re-arms after every character
```

Transmit (chain B), RS-485 driver while sending

```
A8 03 00         LINE: NRZ, read-back pin 3
B8 40 00 00 64   CLOCK: ASYNC, 100 clocks per bit
D8 60            OUT: push-pull while sending, idle 1
18 81 00 00      START: write 1 bit = 0 (start bit)
38 43 00 0A      MAIN: write 10 bits from DATA
```

Per character (first byte of SD1 shown)

```
CF 10   DATA {8,SD1}
C9 03   DATA {2, parity | stop<<1}: parity 1 for 0x10, then stop
78      RUN
```

What comes back. {8,data},{2,parity | stop<<1} per character, so a good character always ends in {2,x} with bit 1 set.

Host work. Check the parity of each character and the telegram's FCS; compute both when sending.

Tested. Request 10 02 01 49 4C 16 received with correct parity; response 10 01 02 00 03 16 sent and decoded with parity checked.

7 Modbus RTU

Modbus RTU frames are binary messages on RS-485, ended by a silence of 3.5 character times and protected by CRC-16/MODBUS sent low byte first. Read Holding Registers from slave 1: 01 03 00

`00 00 02 C4 0B`.

19 200 bit/s, 8E1 (the default for RTU), `DIV` = 2604.

Receive (chain A)

```
A0 00 00          LINE: NRZ on pin 0
B0 40 00 0A 2C    CLOCK: ASYNC, 2604 clocks per bit
10 01 00 00       START: read 1 bit = 0 (start bit) on pin 0
30 00 00 0A       MAIN: read 10 bits (data, parity, stop)
70                RUN: armed, re-arms after every character
```

Transmit (chain B)

```
A8 03 00          LINE: NRZ, read-back pin 3
B8 40 00 0A 2C    CLOCK: ASYNC, 2604 clocks per bit
D8 60             OUT: push-pull while sending, idle 1
18 81 00 00       START: write 1 bit = 0 (start bit)
38 43 00 0A       MAIN: write 10 bits from DATA
```

Receive with the CRC checked on chip (chain A, ISA v0.7)

```
A0 00 00          LINE: NRZ on pin 0
B0 40 00 0A 2C    CLOCK: ASYNC, 19200 bit/s
10 01 00 00       START: read the start bit
21 48 40 00       HDR 1: read 8 data bits, any value, capture
22 41 40 00       HDR 2: read the parity bit, any value, capture
30 00 00 01       MAIN: read the stop bit
F2 F3 80 05       CRC A: 16 bits from HDR 1, init ones, poly 8005 (CRC-16/MODBUS in wire order)
F3 CC 00 00       CRC B: on, check, BYTE (8-bit fields only), SPAN (across frames), residue 0
70                RUN
```

What comes back. Plain receive: per character {`8,data`},{`2,parity|stop<<1`}. With the on-chip check: per character {`8,data`},{`1,parity`},{`1,stop`}, a CRC result {`14,r`} and status `01`; the result after the last character is the message's CRC check (`03` good, `02` bad).

Host work. Use the 3.5-character gap between statuses to split messages. With the on-chip check, rewrite `BRANCH 3` (`F3 CC 00 00`) before each message so the CRC starts afresh, and read the last result. For transmit the host appends CRC-16/MODBUS (init FFFF, reflected, poly 8005, low byte first): the chip's appended CRC would be raw bits without UART framing.

Tested. The request above received (its CRC `C4 0B` recomputed by the test); the response `01 03 04 00 0A 00 0B 9B F6` sent and decoded with parity; with the on-chip check the request ended in result `03`, and with one bit flipped in result `02`.

8 DMX512

DMX512 drives stage lighting over RS-485. A packet is a break (at least 88 µs low), a mark after break (at least 8 µs high), then up to 513 slots sent as 8N2 characters at 250 kbit/s; slot 0 is the start code (`00` for dimmer levels).

250 kbit/s, 8N2, `DIV` = 200. Controller: `uio[3]` to the RS-485 driver. Receiver: `uio[0]` from the RS-485 receiver.

Transmit a packet (chain B): break and mark from headers

```
A8 03 00          LINE: NRZ, read-back pin 3
B8 40 00 00 C8    CLOCK: ASYNC, 200 clocks = 4 us per bit
D8 E0             OUT: push-pull always, idle 1 (mark)
18 81 00 00       START: write 0      } break: 1 + 8 + 8 + 6 = 23 bits
```

```
29 88 00 00      HDR 1: write 8 x 0  }   = 92 us (>= 88 us)
2A 88 00 00      HDR 2: write 8 x 0  }
2B 88 00 C0      HDR 3: write 000000 11: end of break, then 8 us mark after break
38 43 00 37      MAIN: write 55 bits = start code + 4 slots, 11 bits each
```

Per packet

```
C8 00   {1,0}   start bit of the start code
CF 00   {8,00}  start code
CA 03   {3,011} two stop bits, then the next start bit
CF FF   {8,FF}  slot 1 ... and so on
C9 03   {2,11}  last slot: two stop bits only
7A      RUN WRITE: one packet
```

Receive (chain A): UART with a framing check (pattern 3 on page 2)

```
A0 00 00         LINE: NRZ on pin 0
B0 40 00 00 C8   CLOCK: ASYNC, 200 clocks per bit
10 01 00 00      START: read the start bit
21 48 40 00      HDR 1: read 8 bits, any value, capture: the slot
22 01 00 01      HDR 2: read 1 bit, must be 1: first stop bit (a break fails here)
30 00 00 01      MAIN: read the second stop bit
40 08 00 00 01   END: one bit of 1 on pin 0: after an abort, wait for the line to idle
70               RUN: armed
```

What comes back. Transmit: status `01` per packet. Receive: each break returns only status `02` (the stop-bit compare failed and the captured value is dropped); each slot returns `{8,slot},{1,1}` and status `01`. The host restarts its slot counter at every `02`.

Host work. Count slots after each break; nothing else. A full 513-slot universe needs the 3-message-per-slot return traffic to fit in 44 µs per slot, which a `LINK` of 4 clocks per bit does (240 clocks per slot).

Tested. Transmit: break measured 92 µs, mark 8 µs, start code and four slots decoded. Receive: two packets with different break (100 and 140 µs) and mark (12 and 8 µs) lengths, all slots recovered, one `02` per break.

9 LIN

LIN is a single-wire 12 V bus for car body electronics. The master sends a header: a break of at least 13 dominant bits, a delimiter, the sync byte `55` and a protected identifier (6-bit ID plus two parity bits). The node that owns the ID answers with up to 8 data bytes and a checksum ("enhanced": the inverted 8-bit sum with carry wrap-around of the PID and the data).

19 200 bit/s, `DIV` = 2604, open drain into a LIN transceiver (TXD and RXD both on `uio[0]`).

Master: send a frame with data (chain A)

```
A0 00 00         LINE: NRZ, read-back pin 0
B0 40 00 0A 2C   CLOCK: ASYNC, 2604 clocks per bit
D0 A0            OUT: open drain (the LIN transceiver's TXD)
10 81 00 00      START: write 0        } break: 1 + 8 + 4
21 88 00 00      HDR 1: write 8 x 0    }   = 13 dominant bits
22 84 00 00      HDR 2: write 4 x 0    }
23 82 00 01      HDR 3: write 1, 0: break delimiter, then the sync start bit
30 40 00 31      MAIN: write sync, PID, 2 data bytes, checksum (49 bits)
```

Per frame

```
C7 55   {8,55}   sync
C1 01   {2,01}   stop, next start bit
C7 50   {8,50}   PID for frame ID 0x10 (parity bits added by the host)
C1 01   {2,01}
C7 34   {8,34}   data 1
C1 01   {2,01}
C7 12   {8,12}   data 2  (first 8 entries, then RUN, then the rest)
72      RUN WRITE
C1 01   {2,01}
C7 69   {8,69}   enhanced checksum
C0 01   {1,1}    final stop bit
```

Slave: answer frame ID 0x10 on its own (chain A)

```
A0 00 00         LINE: NRZ on pin 0
B0 40 00 0A 2C   CLOCK: ASYNC, 2604 clocks per bit
D0 A0            OUT: open drain
10 08 00 55      START: read 8 bits = 1,0,1,0,1,0,1,0: delimiter, start, sync bits 0-5
21 02 00 01      HDR 1: read sync bits 6, 7 = 1, 0
22 01 00 01      HDR 2: read the sync stop bit = 1
23 01 20 00      HDR 3: seek: wait for the PID start bit, skipping any gap
24 08 00 50      HDR 4: read the PID, must be 50 (otherwise abort and stay silent)
25 01 00 01      HDR 5: read the PID stop bit
30 40 00 1F      MAIN: write the response, 31 bits
C1 01            {2,01}   half a bit of response space, then the start bit
C7 34            {8,34}
C1 01            {2,01}
C7 12            {8,12}
C1 01            {2,01}
C7 69            {8,69}   checksum
C0 01            {1,1}
70               RUN: armed; answers the next header for this ID
```

What comes back. Master: status `01`. Slave: status `01` for each header it answered, `02` for headers addressed to other nodes.

Host work. Compute the PID parity and the checksum. For a slave, reload the response with `DATA` after each answer (or before the next header).

Tested. Master: break measured 13.0 bits, delimiter 1 bit, frame `55 50 34 12 69` decoded with the checksum verified. Slave: a header for ID 0x21 ignored (status `02`), then a header for ID 0x10 with a 2-bit delimiter and a 3-bit gap between sync and PID answered with `34 12 69`.

Notes. The slave's `START` pattern ends at the same bit whatever the delimiter length, because the last delimiter bit is always a 1 followed by the 0 start bit. The seeking `HDR 3` (ISA v0.7) then waits for the PID's start bit, so any inter-byte space the master leaves after the sync byte is skipped.

10 K-Line (ISO 14230 / KWP2000)

K-Line is a single-wire diagnostic bus at 10 400 bit/s. A tester wakes the ECU with the "fast init" pattern, 25 ms low and 25 ms high, and sends StartCommunication straight after (the first bit 50 ms after the first falling edge, ±1 ms). Messages carry a format byte, target, source, service and an 8-bit sum checksum.

10 400 bit/s, `DIV` = 4808; the fast-init wake-up uses `DIV` = 62 500 (1.25 ms bits) and the 5-baud init `DIV` = 2 500 000, which needs the 22-bit divider of ISA v0.7. Both channels drive the K-line open drain: tie `uio[0]` and `uio[3]` to the transceiver's TX, and its RX to the same net.

Fast init: wake-up pattern (chain A)

```
A0 00 00          LINE: NRZ, read-back on the K-line
B0 40 00 F4 24    CLOCK: ASYNC, 62500 clocks = 1.25 ms per bit
D0 A0             OUT: open drain
10 81 00 00       START: write 0         }
21 88 00 00       HDR 1: write 8 x 0     } 20 bits low = 25 ms (TiniL)
22 88 00 00       HDR 2: write 8 x 0     }
23 83 00 00       HDR 3: write 3 x 0     }
24 88 00 FF       HDR 4: write 8 x 1     }
25 88 00 FF       HDR 5: write 8 x 1     } 20 bits high = 25 ms
26 83 00 07       HDR 6: write 3 x 1     }
30 40 00 01       MAIN: write 1 x 1      }
F0 08 00 00       BRANCH 0: read-back rule, keep: status only when the line is free
C0 01             {1,1} for MAIN
72                RUN WRITE: the 50 ms wake-up
```

StartCommunication request (chain B)

```
A8 00 00          LINE: NRZ, read-back on the K-line
B8 40 00 12 C8    CLOCK: ASYNC, 4808 clocks per bit (10 400 bit/s)
D8 A0             OUT: open drain
18 81 00 00       START: write the start bit
38 40 00 09       MAIN: write 8 data bits + stop
CF 81             {8,81}
C8 01             {1,1}
CF EE             {8,EE}
C8 01             {1,1}
CF F0             {8,F0}
C8 01             {1,1}
CF 81             {8,81}
C8 01             {1,1}
78                RUN: sent when chain A's status 01 arrives
CF E0             {8,E0}  checksum, streamed after RUN as the FIFO empties
C8 01             {1,1}
```

Then receive the ECU's answer (chain A)

```
90                CLEAR chain A once the request has gone
A0 00 00
B0 40 00 12 C8
10 01 00 00       UART receive (pattern 1)
30 00 00 09
70
```

5-baud init instead: the address byte at 5 bit/s (chain A)

```
A0 00 00          LINE: NRZ, read-back on the K-line
B0 40 26 25 A0    CLOCK: ASYNC, DIV 2 500 000 = 50 ms: four samples per 5-baud bit
D0 A0             OUT: open drain
10 88 00 F0       START: start bit 0, data bit 0 = 1      (each 4 x)
21 88 00 0F       HDR 1: data bits 1, 2 = 1, 0
22 88 00 F0       HDR 2: data bits 3, 4 = 0, 1
23 88 00 0F       HDR 3: data bits 5, 6 = 1, 0
24 88 00 F0       HDR 4: data bit 7 = 0, stop bit 1   -> address 0x33
30 40 00 01       MAIN: one more sample of idle
F0 08 00 00       BRANCH 0: status when the line is free
C0 01             {1,1}
72                RUN WRITE: the 2-second address byte
```

What comes back. Chain A: status `01` for the wake-up. Chain B: status `01` per request byte. Chain A then returns each response byte as {`8,b`},{`1,1`}.

Host work. Send `RUN` to chain B as soon as chain A's status arrives (about 5 µs later at a fast link); reprogram chain A as a receiver after the request; check the response checksum. P2 (ECU response time) leaves tens of milliseconds for that.

Tested. Fast init: wake-up measured 25.00 ms low; request `81 EE F0 81 E0` starting 50.00 ms after the first falling edge; the ECU model's positive response `80 F0 EE 03 C1 EA 8F 9B` received in full. Message contents follow the example in the EU tachograph regulation (Reg. 3821/85, Annex 1B, Appendix 8). 5-baud init: address `33` sent with correct framing, then the ECU's sync `55` and key bytes `08 08` received at 10 400 bit/s, the tester's inverted key byte `F7` sent, and the ECU's inverted address `CC` received. The 2-second address byte is simulated at 1/40 of real time (`DIV` 62 500) to keep the test short; the 22-bit divider itself is tested at full size by RC5.

Notes. For the 5-baud init, after the address byte reprogram chain A as a 10 400 bit/s receiver (as for fast init), read `55` and the two key bytes, send the inverted second key byte from chain B, and read the ECU's inverted address.

11 SDI-12

SDI-12 connects environmental sensors to a data recorder on one wire at 1200 bit/s, 7 data bits, even parity, 1 stop bit, in negative logic: marking (1) is the low voltage. The recorder wakes the sensors with a break (at least 12 ms of spacing), then at least 8.33 ms of marking, sends a command such as `0M!` (start a measurement on sensor 0) and releases the line; the sensor answers within 15 ms with a line ending in CR LF.

1200 bit/s, 7E1, `DIV` = 41 667; `INV` = 1 on both channels. Tie `uio[0]` and `uio[3]` to the data line (through the usual level shifter to 5 V).

Recorder: break, marking and command (chain B)

```
A8 10 00         LINE: NRZ, inverted (SDI-12 is negative logic), read-back pin 0
B8 40 00 A2 C3   CLOCK: ASYNC, 41667 clocks per bit (1200 bit/s)
D8 60            OUT: push-pull while sending, then release the line
18 81 00 00      START: write 0       } break: 15 spacing bits
29 88 00 00      HDR 1: write 8 x 0   }   = 12.5 ms (>= 12 ms)
2A 86 00 00      HDR 2: write 6 x 0   }
2B 88 00 FF      HDR 3: write 8 x 1   } marking: 10 bits
2C 82 00 03      HDR 4: write 2 x 1   }   = 8.33 ms
38 40 00 1E      MAIN: write 3 characters of 10 bits
F8 08 00 00      BRANCH 0: status only when the line is free
CF 60            {8,60}   start bit + 7 data bits of '0'
C9 02            {2,10}   even parity, stop
CF 9A            {8,9A}   start bit + 7 data bits of 'M'
C9 02            {2,10}   even parity, stop
CF 42            {8,42}   start bit + 7 data bits of '!'
C9 02            {2,10}   even parity, stop
7A               RUN WRITE
```

Receive the answer (chain A)

```
A0 10 00         LINE: NRZ, inverted, pin 0
B0 40 00 A2 C3   CLOCK: ASYNC, 1200 bit/s
10 01 00 00      START: read the start bit
30 00 00 09      MAIN: read 7 data + parity + stop
70               RUN: sent when chain B's status arrives
```

What comes back. Chain B: status `01` once the command has left the pin. Chain A: `{8, data | parity<<7},{1,1}` per character.

Host work. Arm chain A when chain B's status arrives; strip and check the parity bit (bit 7) and assemble the reply up to CR LF.

Tested. Break measured 12.5 ms, marking 8.33 ms, `OM!` decoded with parity; the sensor model's reply `00011<CR><LF>` (sensor 0, 1 value ready in 1 s) received with correct parity.

12 SMBus and PMBus

SMBus is I²C with stricter timing and an optional Packet Error Code: a CRC-8 (poly `07`) over every byte of the transaction, including the address bytes. PMBus (power supplies) uses the same transactions, Write Word and Read Word being the most common.

100 kHz, `DIV` = 250. SDA on `uio[0]`, SCL on `uio[2]`, both with pull-ups.

One program for both transactions (chain A)

```
61              ORDER: MSB first
A0 00 00        LINE: SDA on pin 0
B0 8A 00 00 FA  CLOCK: GENERATE SCL, 100 kHz, read back on pin 2, idle high
D0 A4           OUT: open drain, I2C conditions
50 EA 60        WDOG
10 88 00 80     START: write address 0x40, write (0x80)
21 01 00 00     HDR 1: read ACK = 0
22 88 00 21     HDR 2: write the command code 0x21
23 01 00 00     HDR 3: read ACK
24 98 80 81     HDR 4: read runs only: Sr, address 0x40 read (0x81)
25 11 00 00     HDR 5: read runs only: ACK
30 90 00 18     MAIN: follow the run mode, 3 bytes with ACK slots
```

Write Word

```
C7 34   {8,34}  low byte
C7 12   {8,12}  high byte
C7 CA   {8,CA}  PEC = CRC-8 of 80 21 34 12
72      RUN WRITE: Write Word with PEC
```

Read Word

```
71   RUN READ: Read Word with PEC (low, high, PEC; NACK after the PEC)
```

Read Word with the PEC checked on chip (ISA v0.7): add to the program

```
F2 70 00 07   CRC A: 8 bits from START, init 0, poly 07 (CRC-8)
F3 C8 00 00   CRC B: on, check, BYTE: address, command and data bytes only, not ACKs
71            RUN READ: the result follows the three bytes
```

What comes back. Write: status `01` (a NACK would end the frame with {`0,02`}). Read: {`8,low`},{`8,high`},{`8,PEC`} and status `01`; with the on-chip check also {`14,03`} (good) or {`14,02`} (bad) before the status.

Host work. Compute the PEC for writes (CRC-8 over `80 21 34 12`); the chip's append cannot be used there because the PEC byte needs its own ACK slot. For reads either check it on the host (CRC-8 over `80 21 81 low high PEC` gives 0) or let the chip do it with `BYTE` mode, which skips the ACK bits and the repeated START.

Tested. Write: the device model logged `S 80 A 21 A 34 A 12 A CA A P` and verified the PEC. Read: `S 80 A 21 A Sr 81 A CD A AB A 73 N P`, and the host-side PEC check passed; with the on-chip check the result was `03`, and `02` when the device sent a corrupted PEC.

13 MDIO (Ethernet PHY management)

MDIO (IEEE 802.3 clause 22) reads and writes the 32 registers of an Ethernet PHY. A frame is 32 preamble 1s, start `01`, opcode (`01` write, `10` read), 5-bit PHY address, 5-bit register address, a 2-bit turnaround and 16 data bits. In a read the station releases MDIO for the first turnaround bit and the PHY drives the second to 0.

MDC up to 2.5 MHz, `DIV` = 10. MDIO on `uio[0]` with a pull-up, MDC on `uio[2]`.

Common part (chain A)

```
61              ORDER: MSB first
A0 00 00        LINE: MDIO on pin 0
B0 82 00 00 0A  CLOCK: GENERATE MDC, 2.5 MHz, read back on pin 2
D0 60           OUT: push-pull while sending: MDC push-pull, MDIO released for reads
50 13 88        WDOG
10 88 00 FF     START: preamble 8 x 1  }
21 88 00 FF     HDR 1:  8 x 1          } 32-bit
22 88 00 FF     HDR 2:  8 x 1          } preamble
23 88 00 FF     HDR 3:  8 x 1          }
```

Write a register (PHY 1, register 4)

```
24 88 00 50   HDR 4: ST 01, OP 01 (write), PHYAD[4:1] = 0000
25 88 00 92   HDR 5: PHYAD[0] = 1, REGAD = 00100, TA = 10
30 40 00 10   MAIN: write 16 data bits
C7 01
C7 E1         {8,01},{8,E1}: register 4 <- 01E1
72            RUN WRITE
```

Read a register (PHY 1, register 2): replace HDR 4 and HDR 5, add HDR 6

```
24 88 00 60   HDR 4: ST 01, OP 10 (read), PHYAD[4:1]
25 86 00 22   HDR 5: PHYAD[0], REGAD = 00010 (6 bits)
26 02 00 02   HDR 6: read the turnaround: Z (1), then 0 from the PHY
30 00 00 10   MAIN: read 16 data bits
71            RUN READ
```

What comes back. Write: status `01`. Read: `{8,high},{8,low}` and status `01`. If no PHY answers at that address the turnaround reads 11, `HDR 6` fails, and the frame ends with status `02` and no data.

Host work. Pick the HDR 4 to HDR 6 set for read or write. To go back from a read program to a write program, send `CLEAR` and the write program (a configured `HDR 6` cannot be removed on its own).

Tested. Register 4 written with `01E1`; registers 4 and 2 read back as `01E1` and `0022` from a PHY model that changes MDIO 0 ns after the MDC edge (the IEEE 802.3 minimum), with no clock where the chip and the PHY drove MDIO together; a read from absent PHY 5 aborted cleanly.

Notes. MDC must be push-pull: with a push-pull clock ProEm samples MDIO about one system clock before its own rising edge, so a PHY may change MDIO at the edge itself. With an open-drain clock the chip would sample only after it sees the clock high, which is right for I^2C stretching but too late for a 0 ns PHY. Since ISA v0.7, `DRIVE` = `01` drives MDC push-pull and releases MDIO for the turnaround and the read.

14 SD card in SPI mode

SD cards start in their native mode and enter SPI mode on CMD0 sent with chip select low. Every command is 48 bits: `01`, a 6-bit index, a 32-bit argument, a CRC-7 and an end bit 1. CRCs are only checked for CMD0 and CMD8 by default, but those two must be right. The card answers 0 to 8 bytes later (NCR) with R1 (`01` = idle), or R7 for CMD8.

400 kHz during initialisation, `DIV` = 63. MOSI on `uio[0]` (chain A), SCK on `uio[5]` and CS on `uo[7]` (chain B), MISO on `uio[6]`.

Chain B: clock, chip select and MISO

```
69              ORDER: MSB first
A8 06 00        LINE: MISO on pin 6 (an input-only pad)
B8 85 00 00 3F  CLOCK: GENERATE SCK, 397 kHz, read back on pin 5
D8 C0           OUT: push-pull: SCK on uio[5], card select on uo[7] (active low)
58 4E 20        WDOG
18 80 00 00     START: condition write, sends nothing: the frame starts on RUN
38 06 00 60     MAIN: read 96 bits of MISO (12 bytes), clocking SCK and holding CS low
```

Chain A: MOSI on chain B's clock, with the CRC-7 appended by the chip

```
61              ORDER: MSB first
A0 00 00        LINE: MOSI on pin 0
B0 05 00 00 00  CLOCK: SYNC on chain B's SCK (pin 5), sample rising, change falling
D0 E0           OUT: push-pull, idle 1 (MOSI high between commands)
10 80 00 00     START: condition write, sends nothing
30 40 00 28     MAIN: write the 40-bit command
F2 6E 00 09     CRC A: 7 bits, MAIN only, init 0, poly 09 (CRC-7)
F3 A0 00 00     CRC B: on, append
F4 FC FF FF     REPLY: always, 16 bits of 1 after the CRC: end bit + filler
```

CMD0 (send to both chains)

```
C7 40
C7 00
C7 00
C7 00
C7 00   CMD0, argument 0
72      RUN WRITE (chain A first, so the first bit is ready)
79      RUN on chain B: 96 clocks with CS low
```

What comes back. Chain B returns 12 bytes read during the transaction (`FF` while the card is silent) and status `01`; chain A returns status `01`.

Host work. Find the response as the first byte that is not `FF`. Load the 5 command bytes per command; the chip adds the CRC-7 and the end bit.

Tested. The card model checks the CRC-7 and end bit: CMD0 arrived as `40 00 00 00 00 95` and was answered `01`; CMD8 arrived as `48 00 00 01 AA 87` and was answered with R7 `01 00 00 01 AA`. A wrong CRC would have set bit 3 of R1.

Notes. Chain B owns the clock and the chip select so that one frame covers command, wait and response: chain A's frame ends after the CRC and the filler, but chain B keeps clocking with CS low until it has read 96 bits. This also gives any NCR up to 4 bytes for CMD8 and 5 for CMD0; raise `MAIN` on chain B for slower cards.

15 3-wire SPI

Many sensors (pressure, accelerometers, display drivers) offer a 3-wire SPI mode in which one SDIO line carries data both ways: the host sends an address byte with a read/write bit, then either keeps writing or lets the device drive SDIO for the read.

SCK 1 MHz, `DIV` = 25; SDIO on `uio[0]` with a pull-up, SCK on `uio[2]`, chip select on `uo[6]`.

One program for both directions (chain A)

```
61              ORDER: MSB first
A0 00 00        LINE: SDIO on pin 0
B0 82 00 00 19  CLOCK: GENERATE SCK, 1 MHz, read back on pin 2, mode 0
D0 60           OUT: push-pull while sending; SDIO released for read bits
50 13 88        WDOG
10 80 00 00     START: condition write; chip select on uo[6]
21 A8 00 34     HDR 1: write runs only: address 0x34, R/W bit 0
22 98 00 B4     HDR 2: read runs only: address 0x34 | 0x80
30 80 00 10     MAIN: follow the run mode, 16 bits
```

Write two registers

```
C7 5A
C7 C3   two data bytes
72      RUN WRITE
```

Read two registers

```
71   RUN READ: the chip releases SDIO after the address
```

What comes back. Write: status `01`. Read: {`8,reg`},{`8,reg+1`} and status `01`.

Host work. Choose `RUN READ` or `RUN WRITE`; groups pick the matching address header, as in the I²C example of the ISA document.

Tested. Registers `34`, `35` written with `5A C3` and read back as `9E 07` after the device model changed them; the test counts clocks where the chip and the device both drive SDIO, and there were none.

Notes. Since ISA v0.7 “push-pull while sending” releases the data line for generated read bits and between entries of a generated-clock frame, so SDIO needs no pull-up and runs at full push-pull speed.

16 DALI

DALI (IEC 62386) controls lighting over a two-wire bus that idles high. Bits are Manchester coded at 1200 bit/s with a rising mid-bit edge for 1 (the IEEE 802.3 convention). A forward frame from the controller is a start bit 1, an address byte, a command byte and two stop bits of idle; the gear’s backward frame is a start bit and one byte.

1200 bit/s, `DIV` = 41 666 (Manchester transmit needs an even value). `uio[3]` to the bus driver, `uio[0]` from the bus receiver.

Send a forward frame (chain B)

```
69              ORDER: MSB first
A8 63 00        LINE: Manchester (IEEE convention), read-back pin 3
B8 40 00 A2 C2  CLOCK: ASYNC, 41666 clocks per bit (1200 bit/s, even)
D8 A0           OUT: open drain: the bus idles high, the chip pulls it low
18 81 00 01     START: write the start bit, 1
```

```
38 40 00 10      MAIN: write the 16-bit forward frame
CF 03            {8,03}  address: short address 1, command follows
CF 90            {8,90}  QUERY STATUS
7A               RUN WRITE
```

Receive a backward frame (chain A): Manchester decoder with boundary lock

```
61               ORDER: MSB first
A0 68 00         LINE: Manchester on pin 0, BND: first edge after idle is a bit boundary
B0 40 00 A2 C2   CLOCK: ASYNC, 1200 bit/s
10 01 00 01      START: read the start bit, 1
30 00 00 08      MAIN: read the 8-bit backward frame
70               RUN
```

What comes back. Transmit: status `01`; the line is released within one clock of the last bit, which gives the two idle stop bits. Receive: {`8,frame`} and status `01`.

Host work. Keep the forward-to-backward settling times; nothing else.

Tested. Forward frame `03 90` decoded from the line by a Manchester decoder model; backward frames `A5` and, after idle, `3C` received by the chip's Manchester decoder.

Notes. DALI frames start from the idle-high bus with a falling edge at the start bit's boundary, not in its middle. `BND` (ISA v0.7) tells the decoder so; without it the decoder would lock half a bit out of phase. The earlier workaround, sampling half-bits as NRZ and pairing them on the host, still works for any Manchester frame and also flags coding errors.

17 CAN 2.0B and NMEA 2000

CAN 2.0B extended frames carry a 29-bit identifier: start-of-frame, the 11-bit base ID, SRR and IDE (both recessive), the 18-bit extension, RTR, two reserved bits, the 4-bit length, up to 8 data bytes and the CRC-15, all bit stuffed after five equal bits, then the CRC delimiter, the ACK slot and the end of frame. NMEA 2000 (marine) and J1939 (trucks) use them at 250 kbit/s, with the ID made of priority, PGN and source address; vessel heading is PGN 127250.

250 kbit/s, `DIV` = 200. A CAN transceiver with TXD from `uio[0]` or `uio[3]` and RXD to the same pad; the two channels can share one transceiver.

Receive and acknowledge (chain A)

```
61               ORDER: MSB first
A0 00 A8         LINE: NRZ, unstuff after 5 equal bits (either value)
B0 40 00 00 C8   CLOCK: ASYNC, 200 clocks per bit (250 kbit/s)
D0 80            OUT: open drain (CAN transceiver TXD)
10 01 00 00      START: read the start-of-frame bit
30 00 00 75      MAIN: read 117 bits: ID, control, 8 data bytes, CRC
F2 E0 45 99      CRC A: 15 bits from START, init 0, poly 4599
F3 C0 00 00      CRC B: on, check, residue 0
F4 44 00 02      REPLY: when good, 2 bits, no gap: delimiter 1, ACK 0
70               RUN
```

Transmit with the CRC appended (chain B)

```
69               ORDER: MSB first
A8 03 A8         LINE: NRZ, stuff after 5 equal bits
B8 40 00 00 C8   CLOCK: ASYNC, 250 kbit/s
D8 80            OUT: open drain
18 81 00 00      START: write the start-of-frame bit
```

```
38 40 00 66      MAIN: write 102 bits from DATA
FA E0 45 99      CRC A: as for receive
FB A0 00 00      CRC B: on, append
```

What comes back. Receive: 14 entries of 8 bits and one of 5 (the 117 bits from the base ID to the end of the CRC, MSB first), the CRC result {14,03} and status 01; the ACK is driven in the ACK slot only if the CRC was good. Transmit: status 01.

Host work. Extract the ID and data from the returned bits. To transmit, pack the 102 frame bits after the start-of-frame into DATA entries of up to 8 bits, MSB first: first 8 entries, then RUN WRITE, then the rest as the FIFO empties.

Tested. Receive: a node model sent PGN 127250 from source 0x23 (ID 09F11223) with 8 data bytes, stuffed to 128 bits; every bit and the CRC result came back, and the ACK was dominant in its slot; with one CRC bit flipped the result was 02 and no ACK was sent. Chip to chip: chain B transmitted, chain A checked and acknowledged, and an independent checker on the bus removed the stuff bits and verified the frame, the CRC and the ACK.

18 RC5 infrared

RC5 is the Philips remote-control code: 14 bits of Manchester at 1.778 ms per bit, sent as bursts of a 36 kHz carrier. A frame is two start bits (1, 1), a toggle bit that changes on every key press, a 5-bit address and a 6-bit command, MSB first; a 1 has the carrier burst in the second half of the bit.

1.778 ms bits, DIV = 88 900, above the old 16-bit limit. Receive from a demodulating IR receiver (TSOP-style, output low during a burst) on uio[0]. Transmit: uio[3] is the carrier enable, high during a burst; gate a 36 kHz oscillator with it to drive the IR LED.

Receive (chain A)

```
61               ORDER: MSB first
A0 70 00         LINE: Manchester, inverted (the IR receiver output is active low), pin 0
B0 C0 01 5B 44   CLOCK: RECOVER, 88 900 clocks = 1.778 ms per bit (needs DIV[21:16])
10 02 00 03      START: read the two start bits, 1 1
30 00 00 0C      MAIN: read toggle, 5 address bits, 6 command bits
70               RUN
```

Transmit the envelope (chain B)

```
69               ORDER: MSB first
A8 63 00         LINE: Manchester, pin 3
B8 40 01 5B 44   CLOCK: ASYNC, 1.778 ms per bit
D8 C0            OUT: push-pull always, idle 0: output = carrier enable
18 82 00 03      START: write the two start bits
38 40 00 0C      MAIN: write 12 bits
CF 97            {8,97}: toggle 1, address 00101, command bits 5-4 = 11
CB 05            {4,5}: command bits 3-0 (command 0x35)
7A               RUN WRITE
```

What comes back. Receive: {8,T A4..A0 C5 C4},{4,C3..C0} and status 01. Transmit: status 01.

Host work. Split the 12 bits into toggle, address and command; flip the toggle bit on each new key press when sending. The 36 kHz carrier is outside the chip: ProEm's fastest generated clock is fine for it, but a single channel cannot gate its own clock with its data.

Tested. Address 5, command 0x35, toggle 1 received from a model of the IR receiver output at the real bit time, and the same frame sent as a clean carrier-enable envelope whose 14 bits were decoded from the line.

Notes. The frame starts with a bit whose first half equals the idle level, so the Manchester decoder needs no `BND`; the first edge it sees is a real middle edge.

19 Limits found by the first round of tests, and the fixes

Limit in ISA v0.6	Fix in ISA v0.7	Tested by
`DIV` was 16 bits: nothing slower than 763 bit/s (RC5, K-Line 5-baud init)	`DIV[21:16]` in `CLOCK` byte 2: down to 12 bit/s	RC5 at 88 900 clocks per bit; K-Line 5-baud handshake
Manchester decoder locked onto the first edge, a boundary edge in DALI	`BND` in `LINE`: the first edge after idle is a bit boundary	DALI backward frames received by the decoder
MDIO read by the chip one clock after its open-drain MDC edge	"Push-pull while sending" now releases the data line for reads, so MDC can be push-pull, sampled before the edge	MDIO with a PHY switching 0 ns after MDC
A gap inside a frame shifted the bit count (LIN inter-byte space)	`SEEK` headers skip bits until their first bit appears (18 cells per header)	LIN header with a 3-bit gap
Byte-framed CRCs (Modbus, SMBus PEC) had to stay on the host	`BYTE` (8-bit fields only) and `SPAN` (across frames) in `BRANCH 3`	Modbus and SMBus checks on chip, good and corrupted
"Push-pull while sending" drove during read bits, so shared data lines needed open drain	Data and pair outputs released during generated read bits	3-wire SPI and MDIO with a both-driving check

What remains. The chip appends a CRC as raw bits, so byte-framed transmit CRCs (Modbus, the SMBus PEC byte with its ACK) are still added by the host. RC5 transmit gives the burst envelope; the 36 kHz carrier comes from an external gate. The 5-baud K-Line init is simulated at 1/40 of real time to keep the test short. With an open-drain generated clock the chip still samples after it sees the clock high, which I^2C requires.

20 Running the tests

```
bash scripts/run.sh                 # everything, including t_standards
./build/t_standards                 # all sixteen standards, about 3 minutes
./build/t_standards kline           # one standard: rs485 midi nmea0183 profibus modbus dmx lin
                                    #   kline kline5 sdi12 smbus mdio sdcard spi3 dali can_ext rc5
```

Each line of output names the check, shows what was seen on the line or returned by the chip, and ends in PASS or FAIL. The device models live in the test file: a UART decoder working on the

recorded line, a LIN master and slave, an ECU, an SDI-12 sensor, an SMBus device that logs every byte, an MDIO PHY, an SD card that verifies the CRC-7, a 3-wire SPI device, a DALI decoder and gear, a CAN node with an independent bit-level checker, and an RC5 receiver output.